

Содержание

Введение	7
1. Теоретические основы обработки естественного языка	8
1.1. Математическая постановка задачи языкового моделирования	8
1.2. Эволюция векторных представлений	9
2. Глубокие нейросетевые архитектуры.....	10
2.1. Проблемы рекуррентных нейросетей и механизм внимания	10
2.2. Архитектура Transformer	11
3. Расширение контекста языковых моделей и поисковая генерация	13
3.1. Ограничения параметрической памяти больших языковых моделей	13
3.2. Архитектура поисковой генерации.....	13
4. Парадигма автономных агентов и интеграции инструментов	15
4.1. Концептуальная модель автономного агента	15
4.2. Механизм оркестрации инструментов	15
4.3. Цикл саморефлексии и автономная отладка.....	16
5. Практическая часть	17
5.1. Цель и постановка практической работы	17
5.2. Программно-технический стек.....	17
5.3. Архитектура агента и семантический роутинг	18
5.4. Внешние инструменты и RAG-система.....	18
5.5. Цикл саморефлексии.....	19
5.6. Оценка качества работы системы	19
5.7. Итоги практической части.....	20
Заключение.....	21
Список использованных источников.....	22

Введение

В эпоху стремительного развития искусственного интеллекта происходит масштабная автоматизация рабочих процессов во множестве отраслей, и сфера информационных технологий – не исключение. Сегодня индустрия нуждается в системах, качественно пишущих код и способных брать на себя рутинные инженерные задачи. Критически важно, чтобы искусственный интеллект создавал функционально правильный, безопасный и полностью рабочий продукт.

Классические большие языковые модели, которые только вчера были настоящим прорывом в области искусственного интеллекта, уже не отвечают таким высоким требованиям. Их основная проблема – ограниченность: они не могут сами тестировать написанный код, искать необходимую информацию во внешних источниках, а главное, выстраивать сложные цепочки рассуждений. Кроме того, они в значительной степени подвержены проблеме галлюцинации, а также теряют контекст в ходе длительного диалога.

Для решения данного ряда проблем был предложен совершенно новый подход – автономные агенты искусственного интеллекта. Опираясь на современные алгоритмы обработки естественного языка агенты способны планировать архитектуру, использовать внешние инструменты для поиска необходимой информации, запускать и тестировать код, анализировать ошибки и итеративно доводить решение до идеала.

В данной работе исследуются алгоритмы обработки естественного языка в контексте их применения для построения автономных агентов, а также представлен практический проект – автономный агент для написания кода.

1. Теоретические основы обработки естественного языка

1.1. Математическая постановка задачи языкового моделирования

В основе большинства современных систем обработки естественного языка лежит задача языкового моделирования, которая заключается в моделировании распределения вероятности над последовательностями слов из заданного словаря [1].

Если текст представлен как упорядоченная последовательность токенов $W = (w_1, \dots, w_n)$, каждый из которых принадлежит словарю, то вероятность возникновения данной последовательности формализуется:

$$P(w_1, \dots, w_n) = \prod_{i=1}^n P(w_i | w_1, \dots, w_{i-1})$$

Авторегрессионные языковые модели, используемые для генерации текста, обучаются предсказывать наиболее вероятный следующий токен w_i при условии заданного левого контекста. В процессе обучения модель параметризуется весами Θ , а оптимизация происходит путем минимизации функции отрицательного логарифмического правдоподобия по всему обучающему корпусу D :

$$L(\theta) = - \sum_{W \in D} \sum_{i=1}^{|W|} \log P_{(\theta)}(w_i | w_1, \dots, w_{i-1})$$

В случае разработки автономных агентов контекст $(w_1, \dots, w_{i-1})$ динамически расширяется, включая в себя скрытые системные инструкции, историю диалога, а также логику внутренних рассуждений.

1.2. Эволюция векторных представлений

Алгоритмы машинного обучения не способны напрямую работать с дискретными символьными строками. Фундаментальным этапом любой задачи обработки естественного языка является векторизация текста — математическое отображение токенов в векторное пространство.

Самыми базовыми подходами являются частотные модели, такие как Bag-of-Words и TF-IDF. В модели TF-IDF вес слова w в конкретном тексте d относительно всего корпуса текстов D вычисляется как произведение частоты слова в документе на инвертированную частоту документа. Основная проблема данного подхода — формирование сверхразреженных ортогональных векторов, что впоследствии дает полную потерю информации о семантической близости и синтаксическом порядке.

Продвинутые алгоритмы, такие как Word2Vec, используя идею определения лингвистического значения по контекстуальному окружению, проецируют токены в непрерывное пространство низкой размерности $\mathbb{R}^d$. Например, в одном из подходов Skip-Gram целевая функция максимизирует вероятность предсказания контекстных слов w_{t+j} по заданному центральному слову w_t в рамках скользящего окна размера c :

$$J(\theta) = \frac{1}{T} \sum_{t=1}^T \sum_{-c \leq j \leq c, j \neq 0} \log P(w_{t+j} | w_t)$$

Однако данные методы обладают критическим ограничением — статичностью представления: каждому слову словаря ставится в соответствие ровно один вектор вне зависимости от контекста. Именно эта потребность в динамическом пересчёте весовых коэффициентов побудила переход к глубоким архитектурам [1].

2. Глубокие нейросетевые архитектуры

2.1. Проблемы рекуррентных нейросетей и механизм внимания

Долгое время основными алгоритмами обработки последовательностей были рекуррентные нейронные сети и их модификация с долгой краткосрочной памятью. Их фундаментальный недостаток заключался в строгой последовательности вычислений: каждый следующий токен обрабатывался только после предыдущего. Это делало невозможным распараллеливание процессов, а на длинных контекстах приводило к затуханию градиентов и критической потере информации.

Решением этой проблемы стал механизм внутреннего внимания. В отличие от рекуррентных подходов, он обрабатывает всю входную последовательность одновременно, позволяя каждому токenu напрямую взаимодействовать с любым другим. Для вычисления весов внимания каждый токен линейно проецируется в три вектора, образующих матрицы: запросов (Q — какую информацию токен «ищет»), ключей (K — какую информацию токен «предлагает») и значений (V — семантическое содержание токена). Функция масштабированного скалярного произведения вычисляет матрицу внимания как [2]:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Матричное умножение QK^T определяет степень попарной релевантности токенов. Деление на корень из размерности ключа $\sqrt{d_k}$ выступает стабилизатором: оно масштабирует значения, предотвращая попадание функции $softmax$ в области с околонулевыми градиентами.

2.2. Архитектура Transformer

В 2017 году на базе механизма внимания была предложена архитектура Transformer, в которой полностью отсутствуют рекуррентные слои. Отказ от рекуррентности позволилкратно ускорить вычисления, однако привел к новой проблеме: нейросеть стала воспринимать входной текст как неупорядоченное множество слов. Этот недостаток компенсируется слоем позиционного кодирования, который интегрирует в векторное представление каждого токена уникальный гармонический сигнал:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$
$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

Где pos — позиция токена в последовательности, а i — индекс измерения вектора. Благодаря данному механизму модель получает способность вычислять относительные и абсолютные расстояния между элементами текста, что критически важно при анализе синтаксической структуры.

Сердцем архитектуры Transformer является блок многоголового внимания Multi-Head Attention. Он развивает базовую концепцию механизма внимания: вместо одного прохода вычислений входные данные параллельно проецируются в несколько независимых подпространств («голов»), в каждом из которых вычисляется собственное внимание, после чего результаты конкатенируются.

Такая архитектура позволяет нейросети одновременно фокусироваться на различных аспектах контекста: например, одна «голова» может отслеживать синтаксическую вложенность, а другая — семантические зависимости. Помимо блоков внимания, каждый слой Transformer включает классическую

полносвязную нейронную сеть. Для стабилизации градиентов в процессе обучения применяются механизмы нормализации слоя и остаточные связи [2].

Оригинальная архитектура Transformer состояла из двух связанных модулей: энкодера и декодера. Однако в дальнейшем развитие индустрии разделилось на два обособленных направления: Encoder-only и Decoder-only модели. Первые специализируются на двунаправленном понимании текста, извлечении эмбеддингов и классификации. Вторые, работающие по принципу авторегрессии, предназначены исключительно для генерации последовательностей. Именно Decoder-only модели стали стандартом в задачах кодогенерации и легли в основу современных интеллектуальных агентов.

3. Расширение контекста языковых моделей и поисковая генерация

3.1. Ограничения параметрической памяти больших языковых моделей

Несмотря на то, что авторегрессионные Decoder-only модели стали стандартом в задачах генерации кода, они обладают фундаментальным ограничением, связанным с архитектурой их памяти. Вся информация, усвоенная моделью на этапе предварительного обучения, остается «замороженной» в виде статичных числовых значений в матрицах весов.

В контексте программной инженерии это порождает две критические проблемы. Первая - неактуальность знаний: модель не располагает сведениями о новых библиотеках, фреймворках, появившихся после завершения ее обучения. Вторая — проблема ограничения контекстного окна: современные проекты содержат сотни тысяч строк кода. Поместить весь репозиторий в оперативный контекст модели технически невозможно из-за квадратичной вычислительной сложности механизма внимания.

Если попытаться заставить модель написать функцию, использующую внутреннюю архитектуру неизвестного ей проекта, она с высокой вероятностью сгенерирует синтаксически верный, но логически ошибочный код. Это явление называется галлюцинацией.

3.2. Архитектура поисковой генерации

Для преодоления информационной изоляции языковых моделей был разработан архитектурный подход поисковой генерации - Retrieval-Augmented Generation (RAG). Данный паттерн объединяет вычислительную мощность параметрической памяти нейросети с динамически обновляемой внешней памятью, в роли которой выступают векторные базы данных, содержащие актуальную документацию или исходный код локального проекта [3].

Архитектура RAG структурно разделена на два независимых компонента: поисковый модуль (ретривер) и модуль синтеза (генератор). Пайплайн работы алгоритма состоит из трех ключевых этапов:

1. Индексация кодовой базы. Исходный код репозитория и сопутствующая документация предварительно разбиваются на семантические фрагменты (чанки). Затем каждый фрагмент пропускается через специализированную модель-энкодер, преобразуется в плотное векторное представление (эмбединг) и сохраняется в векторную базу данных.

2. Векторный поиск. При поступлении пользовательского запроса q , он подвергается аналогичной векторизации. Ретривер осуществляет поиск k наиболее релевантных фрагментов d в базе данных, максимизируя метрику косинусного сходства между векторами.

3. Аугментация и генерация. Извлеченные фрагменты кода интегрируются в системную инструкцию (промпт) совместно с исходным запросом пользователя. Таким образом, генератор получает семантически обогащенный контекст и синтезирует технически достоверный ответ [3].

В контексте автоматизации написания кода RAG наделяет модель способностью к «пассивному чтению» - быстрому изучению нового материала, например новой версии библиотеки [3].

4. Парадигма автономных агентов и интеграции инструментов

4.1. Концептуальная модель автономного агента

Эволюция систем на базе алгоритмов глубокого обучения привела к качественному сдвигу: переходу от пассивных диалоговых моделей к парадигме автономных агентов. В данной концепции языковая модель выполняет роль центрального когнитивного ядра, способного не только генерировать текст, но и воспринимать текущее состояние вычислительной среды, выстраивать долгосрочные планы и активно изменять эту среду [5].

Фундаментальным паттерном построения подобных систем стала архитектура ReAct (Reasoning and Acting). Если традиционные подходы предполагали либо изолированную генерацию рассуждений (как в методах Chain-of-Thought), либо слепое предсказание действий, то парадигма ReAct синхронизирует оба этих процесса. Модель функционирует в рамках строгого итеративного цикла [4]:

- Наблюдение (Observation): сбор актуальных данных о состоянии системы (например, анализ структуры локальной директории или получение результатов выполнения скрипта).
- Мысль (Thought): внутренняя генерация логического вывода, в ходе которой нейросеть анализирует наблюдение и принимает решение о следующем шаге.
- Действие (Action): вызов конкретного внешнего инструмента с вычисленными аргументами.

Такой подход позволяет агенту декомпозировать комплексные инженерные задачи на последовательность элементарных шагов, непрерывно корректируя свою траекторию на основе обратной связи от среды [4].

4.2. Механизм оркестрации инструментов

Для преодоления барьера между генерацией слов и выполнением реальных команд применяется механизм вызова функций (Tool Calling). Доступные агенту

программные инструменты (API, скрипты, консольные команды) формализуются в виде строгих JSON-схем, описывающих сигнатуры функций. Эти схемы передаются на вход модели вместе с системной инструкцией [5, 6].

Современные Decoder-only модели проходят специализированный этап тонкой настройки (Fine-tuning), обучаясь сопоставлять пользовательские намерения с предоставленными схемами. В результате, когда агент принимает решение о необходимости физического действия, он генерирует не связный текст на естественном языке, а строго структурированный JSON-объект [5, 6].

4.3. Цикл саморефлексии и автономная отладка

Критически важным аспектом автономной работы агента является способность системы выявлять и исправлять собственные ошибки - механизм саморефлексии (Self-Reflection). Практика показывает, что даже передовые языковые модели склонны допускать синтаксические опечатки или пытаться импортировать несуществующие библиотеки.

В парадигме агента процесс отладки замыкается в автоматический контур: при тестовом запуске скрипта среда выполнения перехватывает потоки стандартного вывода (STDOUT) и ошибок (STDERR).

В случае аварийного завершения программы агент получает лог ошибки (Traceback) в качестве нового системного наблюдения. Применяя алгоритмы семантического анализа, когнитивное ядро расшифровывает текст ошибки, сопоставляет его с контекстом ранее написанного кода, формулирует корректирующую мысль и инициирует новое действие — перезапись файла с исправленным участком кода.

Именно эта совокупность авторегрессионного моделирования, поисковой генерации, паттернов логического рассуждения и прямого доступа к операционной системе позволяет создавать интеллектуальных агентов, способных решать инженерные задачи без непрерывного контроля человека.

5. Практическая часть

5.1. Цель и постановка практической работы

Целью практической части являлась разработка MarceCode - интеллектуального агента для написания, отладки и тестирования программного кода на основе современных алгоритмов обработки естественного языка, фреймворка LangGraph и парадигмы ReAct.

В рамках практики решалась прикладная задача создания графовой агентной системы, способной вести осмысленный диалог с пользователем, самостоятельно искать необходимые алгоритмы в локальной кодовой базе или интернете, писать код и безопасно тестировать его в изолированном окружении.

Для достижения поставленной цели были определены следующие задачи:

1. Спроектировать архитектуру конечного автомата для управления поведением агента.
2. Внедрить векторную базу данных для реализации механизмов поисковой генерации.
3. Разработать систему семантического роутинга для разделения простых диалоговых запросов и сложных инженерных задач.
4. Разработать изолированную среду выполнения на базе технологии контейнеризации для безопасного запуска сгенерированного кода.
5. Провести автоматизированную оценку агента по метрикам Pass@1 и Routing Accuracy.

5.2. Программно-технический стек

Система разработана на языке программирования Python (версия 3.10+) с применением современных фреймворков. В ходе реализации использовались следующие библиотеки и технологии:

1. Оркестрация агента: библиотека LangGraph для декларативного описания узлов и переходов [7].

2. Векторный поиск: СУБД Qdrant в связке с моделью эмбедингов multilingual-MiniLM, интегрированной через библиотеку fastembed [8].
3. Изолированная среда: Docker SDK для создания и управления легковесными Python-контейнерами.
4. Внешние интеграции: ChatGroq для использования большой языковой модели Llama-3.3-70b-Versalite и поисковый API Tavily для Web-поиска.
5. Пользовательский интерфейс: графический интерфейс реализован средствами фреймворка Streamlit.
6. Оценка и тестирование: библиотека pytest для unit-тестов.

5.3. Архитектура агента и семантический роутинг

Ядро системы MarceCode реализовано как ориентированный граф вычислений. В основе лежит структура данных AgentState (типизированное состояние), которая хранит историю сообщений, промежуточные шаги и результаты работы инструментов, и флаги состояний.

Для оптимизации вычислительных ресурсов большой языковой модели внедрен семантический роутинг. На этапе анализа запроса агент классифицирует намерение (интент) пользователя. Если запрос представляет собой обычную беседу, граф направляет его в легковесный диалоговый узел, минуя ресурсоемкие блоки RAG и кодогенерации. Если запрос связан непосредственно с написанием программы, активируется полноценный цикл ReAct.

5.4. Внешние инструменты и RAG-система

Интеграция языковой модели с внешней средой реализована через механизм вызова инструментов. Агенту MarceCode предоставлены три ключевых инструмента:

1. Web Search: инструмент поиска информации в интернете через API Tavily. Позволяет агенту находить актуальную документацию по библиотекам.

2. Code Search: инструмент взаимодействия с локальной базой знаний. Для улучшения качества поиска применяется метод Hypothetical Document Embeddings - генерация гипотетического ответа перед векторным поиском, а алгоритм Corrective RAG верифицирует извлеченные фрагменты кода на предмет галлюцинаций, отсекая нерелевантную информацию [3, 8].

3. Secure Sandbox: инструмент безопасного выполнения кода. Модуль динамически поднимает изолированный Docker-контейнер, выполняет в нем сгенерированный агентом Python-скрипт и перехватывает STDOUT и STDERR. Инструмент защищен таймаутами от бесконечных циклов и изолирован от основной файловой системы сервера.

5.5. Цикл саморефлексии

В случае, если код, написанный агентом, падает в Sandbox-контейнере (например, выбрасывает SyntaxError или не проходит встроенные логические assert), перехваченный Traceback направляется обратно в граф.

Благодаря механизмам обработки естественного языка, агент воспринимает этот лог как новое наблюдение. На следующем шаге графа модель анализирует причину сбоя, формирует корректирующий план и генерирует обновленный патч кода. Этот цикл продолжается до тех пор, пока выполнение внутри Sandbox не вернет код успеха.

5.6. Оценка качества работы системы

Оценка работоспособности компонентов MarceCode была разделена на два этапа. На первом этапе были реализованы модульные тесты для инструментов: проверка работы Docker sandbox (а именно отработка синтаксических ошибок и бесконечных циклов), тестирование поискового клиента через Mock-объекты и проверка порога релевантности для Qdrant.

На втором этапе проводилась оценка интеллектуального ядра на заранее подготовленном наборе эталонных запросов Golden dataset. Были измерены следующие метрики:

- Routing Accuracy (Точность маршрутизации): 100% - система безошибочно разделяет обычную беседу и технические задачи.
- CRAG/Fallback Precision (Точность фильтрации контента): 100% - система корректно оценивает релевантность найденных в RAG кусков кода, отсекая мусор и предотвращая галлюцинации.
- Pass@1 Execution Rate (Успешность с первой попытки): 100% - агент успешно генерирует рабочий, синтаксически верный код с первой попытки в заданных условиях.

5.7. Итоги практической части

В ходе практической работы спроектирован и реализован полнофункциональный автономный агент MarceCode. Применение передовых архитектурных подходов, таких как графы состояний, семантический роутинг, продвинутая RAG-система и изолированные среды тестирования, позволило создать устойчивую систему для автономной разработки.

Заключение

В ходе выполнения данной практической работы было проведено комплексное исследование алгоритмов обработки естественного языка в контексте их применения для решения сложных инженерных задач и автоматизации генерации программного кода.

Анализ теоретической базы показал, что переход от классических частотных моделей к механизму внимания позволил вычислительным системам преодолеть проблему понимания контекста. Современные авторегрессионные языковые модели способны не только генерировать связный текст, но и анализировать строгие синтаксические структуры формальных языков программирования.

Однако исследование выявило критические ограничения языковых моделей: изолированность от внешней среды, неактуальность параметрической памяти и подверженность галлюцинациям. Решением этих проблем стали интеграция паттернов рассуждения, механизмов вызова функций и циклов саморефлексии.

Практическая значимость работы подтверждена успешной реализацией и тестированием автономной программной системы MarceCode. Разработанный агент, построенный на базе графа состояний, продемонстрировал способность корректно классифицировать запросы, работать с локальной базой знаний и безопасно выполнять сгенерированный код в изолированном контейнере.

Автоматизированная оценка качества показала 100% эффективность системы на эталонном наборе данных по метрикам Routing Accuracy и Pass@1 Execution Rate. Таким образом, все поставленные задачи были выполнены, а главная цель работы — создание интеллектуальной системы для автономного программирования — успешно достигнута.

Список использованных источников

1. Jurafsky, D. Speech and Language Processing / D. Jurafsky, J. H. Martin. — 3rd ed. draft. — Pearson, 2024. — 624 p.
2. Vaswani, A. Attention Is All You Need / A. Vaswani, N. Shazeer, N. Parmar [et al.] // Advances in Neural Information Processing Systems. — 2017. — Vol. 30. — P. 5998–6008.
3. Lewis, P. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks / P. Lewis, E. Perez, A. Piktus [et al.] // Advances in Neural Information Processing Systems. — 2020. — Vol. 33. — P. 9459–9474.
4. Yao, S. ReAct: Synergizing Reasoning and Acting in Language Models / S. Yao, J. Zhao, D. Yu [et al.] // International Conference on Learning Representations (ICLR). — 2023.
5. Wang, L. A Survey on Large Language Model based Autonomous Agents / L. Wang, C. Ma, X. Feng [et al.] // Frontiers of Computer Science. — 2024. — Vol. 18. — P. 1–26.
6. Schick, T. Toolformer: Language Models Can Teach Themselves to Use Tools / T. Schick, J. Dwivedi-Yu, R. Dessì [et al.] // Advances in Neural Information Processing Systems. — 2024. — Vol. 36.
7. LangGraph: Framework for building stateful, multi-actor applications with LLMs [Электронный ресурс] // LangChain Official Documentation. — Режим доступа: <https://python.langchain.com/docs/langgraph> (дата обращения: 14.05.2026).
8. Qdrant: Vector Database for next-generation AI applications [Электронный ресурс] // Qdrant Official Documentation. — Режим доступа: <https://qdrant.tech/documentation> (дата обращения: 14.05.2026).

Приложение А

Фрагменты исходного кода MarceCode.

```
1 import docker, time
2 from typing import Dict, Any
3
4 class DockerSandbox:
5     def __init__(self, image: str="python:3.10-slim"):
6         self.client = docker.from_env()
7         self.image = image
8
9         try:
10             self.client.images.get(self.image)
11         except docker.errors.ImageNotFound:
12             print(f"Pulling {self.image}...")
13             self.client.images.pull(self.image)
14
15     def execute_code(self, code: str, timeout: int=10) -> Dict[str, Any]:
16         container=None
17         try:
18             container = self.client.containers.run(
19                 self.image,
20                 command=["python", "-c", code],
21                 detach=True,
22                 mem_limit="128m",
23                 nano_cpus=500_000_000,
24                 pids_limit=64,
25                 network_disabled=True,
26                 read_only=True,
27                 tmpfs={"/tmp": "rw,noexec,nosuid,size=64m"},
28                 security_opt=["no-new-privileges"],
29                 user="1000:1000",
30                 cap_drop=["ALL"]
31             )
32
33             starttime = time.time()
34
35             while container.status in ["created", "running"]:
36                 container.reload()
37                 if time.time() - starttime > timeout:
38                     container.kill()
39                     return {
40                         "status": "error",
41                         "stdout": "",
42                         "stderr": f"TimeoutError: Code execution exceeded {timeout} seconds.",
43                         "exit_code": 124
44                     }
45                 time.sleep(0.1)
46
47             res = container.wait()
48
49             stdout = container.logs(stdout=True, stderr=False).decode("utf-8")
50             stderr = container.logs(stdout=False, stderr=True).decode("utf-8")
51
52             return {
53                 "status": "success" if res["StatusCode"]==0 else "error",
54                 "stdout": stdout.strip(),
55                 "stderr": stderr.strip(),
56                 "exit_code": res["StatusCode"]
57             }
58         except Exception as e:
59             return {
60                 "status": "error",
61                 "stdout": "",
62                 "stderr": str(e),
63                 "exit_code": -1
64             }
65         finally:
66             if container:
67                 try:
68                     container.remove(force=True)
69                 except: pass
```

Рисунок А.1 – Реализация инструмента запуска и тестирования кода в изолированной Docker-среде

```

1  import os, json
2  from tavily import TavilyClient
3
4  class WebSearchTool:
5      def __init__(self):
6          self.client = TavilyClient(api_key=os.getenv("TAVILY_API_KEY"))
7
8      def search(self, query: str, max_res: int = 3) -> str:
9          try:
10             response = self.client.search(query=query, max_results=max_res)
11             items = []
12             for r in response.get("results", []):
13                 items.append({
14                     "title": r.get("title", ""),
15                     "content": r.get("content", ""),
16                     "url": r.get("url", "")
17                 })
18
19             if not items:
20                 return json.dumps({
21                     "status": "empty",
22                     "source": "web",
23                     "query": query,
24                     "results": []
25                 }, ensure_ascii=False)
26
27             return json.dumps({
28                 "status": "ok",
29                 "source": "web",
30                 "query": query,
31                 "results": items
32             }, ensure_ascii=False)
33
34         except Exception as e:
35             return json.dumps({
36                 "status": "error",
37                 "source": "web",
38                 "query": query,
39                 "error": str(e)
40             }, ensure_ascii=False)

```

Рисунок А.2 – Реализация инструмента веб-поиска

```

1  import json
2  from qdrant_client import QdrantClient
3
4  class CodebaseSearchTool:
5      def __init__(self, collection_name: str="repo_codebase_v2", path: str="./qdrant_local_db"):
6          self.client = QdrantClient(path=path)
7          self.collection_name = collection_name
8
9          self.client.set_model("sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2")
10
11         if not self.client.collection_exists(collection_name):
12             self.client.create_collection(
13                 collection_name=collection_name,
14                 vectors_config=self.client.get_fastembed_vector_params()
15             )
16
17     def add_code_chunk(self, chunks: list[dict]):
18         self.client.add(
19             collection_name=self.collection_name,
20             documents=[chunk["text"] for chunk in chunks],
21             metadata=[chunk["metadata"] for chunk in chunks],
22             ids=[chunk["id"] for chunk in chunks]
23         )
24
25     def search(self, query: str, limit: int=5, min_score: float=0.50) -> str:
26         try:
27             res = self.client.query(
28                 collection_name=self.collection_name,
29                 query_text=query,
30                 limit=limit
31             )
32
33             points = res or []
34             if not points:
35                 return json.dumps({
36                     "status": "empty",
37                     "source": "local_codebase",
38                     "query": query,
39                     "results": []
40                 }, ensure_ascii=False)
41
42             formatted = []
43             for p in points:
44                 score = getattr(p, "score", 0.0) or 0.0
45                 if score < min_score:
46                     continue
47
48                 payload = getattr(p, "metadata", getattr(p, "payload", {})) or {}
49
50                 formatted.append({
51                     "score": float(score),
52                     "file": payload.get("file", "unknown_file"),
53                     "text": payload.get("document", payload.get("text", ""))
54                 })
55
56             if not formatted:
57                 return json.dumps({
58                     "status": "low_confidence",
59                     "source": "local_codebase",
60                     "query": query,
61                     "results": []
62                 }, ensure_ascii=False)
63
64             return json.dumps({
65                 "status": "ok",
66                 "source": "local_codebase",
67                 "query": query,
68                 "results": formatted
69             }, ensure_ascii=False)
70
71         except Exception as e:
72             return json.dumps({
73                 "status": "error",
74                 "source": "local_codebase",
75                 "query": query,
76                 "error": str(e)
77             }, ensure_ascii=False)

```

Рисунок А.3 – Реализация инструмента поиска в локальной базе

```

1  from typing import Optional, Literal
2  from typing_extensions import TypedDict
3  from pydantic import BaseModel, Field
4
5  class GradeDocuments(BaseModel):
6      |   score: str = Field(description="Оценка релевантности: 'yes' или 'no'")
7
8  class IntentResult(BaseModel):
9      |   intent: Literal["smalltalk", "coding", "other"] = Field(description="Тип сообщения")
10
11 class AgentState(TypedDict):
12     |   user_input: str
13     |   intent: str
14     |   context: str
15     |   source_used: str
16     |   generated_answer: str
17     |   code_to_execute: Optional[str]
18     |   execution_result: str
19     |   final_output: str

```

Рисунок А.4 – Определение структур данных и типизированного состояния агента

```

1  import re
2  from typing import Optional
3
4  def extract_code_block(text: str) -> Optional[str]:
5      |   m = re.search(r"```python\s*(.*?)```", text, flags=re.DOTALL | re.IGNORECASE)
6      |   if m: return m.group(1).strip()
7      |   m2 = re.search(r"```(.*?)```", text, flags=re.DOTALL)
8      |   return m2.group(1).strip() if m2 else text.strip()
9
10 def wants_web(text: str) -> bool:
11     |   return any(k in text.lower() for k in ["в интернете", "в инете", "веб", "web", "internet", "онлайн"])

```

Рисунок А.5 - Вспомогательные функции извлечения кода и парсинга запросов

```

1  import json
2  from langchain_core.messages import HumanMessage, SystemMessage
3
4  from config import llm, hyde_llm, code_db, web_search_tool, sandbox
5  from agent.state import AgentState, GradeDocuments, IntentResult
6  from agent.utils import extract_code_block, wants_web
7
8  evaluator_llm = llm.with_structured_output(GradeDocuments)
9  intent_llm = llm.with_structured_output(IntentResult)
10
11 def classify_node(state: AgentState):
12     t = state["user_input"].strip().lower()
13     obvious_smalltalk = ["привет", "здравствуй", "спасибо", "пока", "как дела", "лол", "кек", "отлично"]
14
15     if any(x in t for x in obvious_smalltalk) and len(t.split()) <= 10:
16         return {"intent": "smalltalk"}
17
18     prompt = (
19         "Классифицируй сообщение пользователя в один класс:\n"
20         "- smalltalk: эмоции, приветствия, благодарности\n"
21         "- coding: запрос написать/найти/исправить/объяснить код\n"
22         "- other: всё остальное\n\n"
23         f"Сообщение: {state['user_input']}"
24     )
25     try:
26         res = intent_llm.invoke(prompt)
27         return {"intent": res.intent}
28     except: return {"intent": "other"}
29
30 def smalltalk_node(state: AgentState):
31     res = llm.invoke([
32         SystemMessage(content="Ответь коротко и дружелюбно, без инструментов."),
33         HumanMessage(content=state["user_input"])
34     ])
35     return {"final_output": res.content}
36
37 def other_node(state: AgentState):
38     return {"final_output": "Понял. Если хочешь, могу помочь  кодом."}
39

```

Рисунок А.6 - Реализация узлов семантического роутинга и классификации намерений

```

def retrieve_node(state: AgentState):
    query = state["user_input"]
    context = ""
    source_used = "unknown"

    if wants_web(query):
        print("\n[Node: Retrieve] -> Ищу в интернете")
        context = web_search_tool.search(query)
        source_used = "internet"
    else:
        print("\n[Node: Retrieve] -> Ищу локально (HyDE + Qdrant)")

        hyde_prompt = f"Write a brief hypothetical Python code snippet for the query: '{query}'. Return ONLY python code."
        hypo_resp = hyde_llm.invoke(hyde_prompt).content
        hypo_code = extract_code_block(hypo_resp) or hypo_resp

        search_query = f"{query}\n{hypo_code}"
        local_context_json = code_db.search(search_query, limit=5, min_score=0.50)

        is_relevant = False
        try:
            parsed = json.loads(local_context_json)
            if parsed.get("status") == "ok":
                found_blocks = parsed.get("results", [])
                blocks_text = "\n\n".join([f"File: {b['file']}\nCode:\n{b['text']}" for b in found_blocks])

                eval_prompt = (
                    f"Query: {query}\n\n"
                    f"Retrieved Code:\n{blocks_text}\n\n"
                    f"Does this code contain relevant logic or algorithms to answer the user's specific query? "
                    f"Answer strictly 'yes' or 'no'."
                )
                grade = evaluator_llm.invoke(eval_prompt)
                if grade.score.lower() == "yes":
                    is_relevant = True
        except Exception:
            pass

        if is_relevant:
            print("[Node: Retrieve] -> Evaluator ОДОБРИЛ локальный код")
            context = local_context_json
            source_used = "local_db"
        else:
            print("[Node: Retrieve] -> Локально пусто или Evaluator ОТКЛОНИЛ. Делаю Fallback в Web")
            web_context = web_search_tool.search(query)
            context = f"LOCAL DB RESULT: {local_context_json}\n\n--- WEB FALLBACK ---\n{web_context}"
            source_used = "local_db+internet"

    return {"context": context, "source_used": source_used}

```

Рисунок А.7 - Реализация узла векторного поиска с применением алгоритмов HyDE и CRAG

```

def generate_node(state: AgentState):
    print("\n[Node: Generate] -> Пишу ответ и код")
    prompt = (
        "Ты AI-ассистент по программированию.\n"
        "Пользователь запросил код. Используй контекст ниже.\n"
        "Верни ответ СТРОГО в следующем формате:\n\n"
        "Краткое объяснение логики.\n\n"
        "```python\n"
        "# Твой код здесь\n"
        "```\n\n"
        "ВАЖНО: НЕ пиши строку 'Source used:'. Я добавлю её сам в интерфейсе.\n"
        "КАТЕГОРИЧЕСКИ ЗАПРЕЩЕНО использовать функцию input()! "
        "Всегда используй жестко заданные переменные и выводи результаты через print().\n\n"
        f"Запрос пользователя:\n{state['user_input']}\n\n"
        f"Контекст:\n{state['context']}\n\n"
    )
    res = llm.invoke([HumanMessage(content=prompt)])
    answer = res.content

    code = extract_code_block(answer)
    return {"generated_answer": answer, "code_to_execute": code}

def execute_node(state: AgentState):
    code = state.get("code_to_execute")
    if code:
        print("\n[Node: Execute] -> Запускаю код в Sandbox")
        res = sandbox.execute_code(code)
        return {"execution_result": res, "final_output": state["generated_answer"]}
    return {"final_output": state["generated_answer"]}

```

Рисунок А.8 - Реализация узлов генерации программного кода и вызова среды выполнения

```

from langgraph.graph import END
from agent.state import AgentState

def route_after_classification(state: AgentState) -> str:
    intent = state["intent"]
    if intent == "smalltalk": return "smalltalk"
    if intent == "other": return "other"
    return "retrieve"

def route_after_generation(state: AgentState) -> str:
    if state.get("code_to_execute"):
        return "execute"
    return END

```

Рисунок А.9 - Логика условных переходов в графе состояний


```

1 from langgraph.graph import StateGraph, START, END
2
3 from agent.state import AgentState
4 from agent.nodes import classify_node, smalltalk_node, other_node, retrieve_node, generate_node, execute_node
5 from agent.edges import route_after_classification, route_after_generation
6
7
8 workflow = StateGraph(AgentState)
9
10 workflow.add_node("classify", classify_node)
11 workflow.add_node("smalltalk", smalltalk_node)
12 workflow.add_node("other", other_node)
13 workflow.add_node("retrieve", retrieve_node)
14 workflow.add_node("generate", generate_node)
15 workflow.add_node("execute", execute_node)
16
17 workflow.add_edge(START, "classify")
18
19 workflow.add_conditional_edges(
20     "classify",
21     route_after_classification,
22     {"smalltalk": "smalltalk", "other": "other", "retrieve": "retrieve"}
23 )
24
25 workflow.add_edge("smalltalk", END)
26 workflow.add_edge("other", END)
27 workflow.add_edge("retrieve", "generate")
28
29 workflow.add_conditional_edges(
30     "generate",
31     route_after_generation,
32     {"execute": "execute", "END": END}
33 )
34
35 workflow.add_edge("execute", END)
36
37 app = workflow.compile()

```

Рисунок А.10 – Реализация графа состояний